

Lab3: Thread 实验报告

李睿阳 524120910059

1. Uthread

实现思路

对于每个线程，储存栈空间/栈地址/函数入口/寄存器/运行情况（FREE, RUNNING, RUNNABLE）。在线程切换时交换寄存器/程序入口/栈地址。

修改的文件

- `user/uthread.c`
- `user/uthread_switch.S`

code

在 `user/uthread.c` 的 `thread_create` 函数中，把函数入口地址赋值给 `ra`（返回地址），栈指针 `sp` 指向分配给该线程的栈的顶部：

```
void thread_create(void (*func)())
{
    // .....
    t->context.ra = (uint64)func;
    t->context.sp = (uint64)t->stack + STACK_SIZE;
}
```

在 `thread_schedule` 中的线程切换：

```
thread_switch((uint64)&t->context, (uint64)&current_thread->context);
```

在 `user/uthread_switch.S` 中，保存和恢复 callee-saved 的寄存器：

```
thread_switch:
    sd ra, 0(a0)
    sd sp, 8(a0)
    sd s0, 16(a0)
    /* ..... */
    sd s11, 104(a0)

    ld ra, 0(a1)
    ld sp, 8(a1)
    ld s0, 16(a1)
    /* ..... */
```

```
ld s11, 104(a1)
ret
```

调试过程中遇到的问题

1. 注意把 `stack` 的指针存储到 `context.sp` 中，并加上栈的默认大小作为初始栈底地址。
2. 在线程切换的汇编代码中，只需要保存 **callee-saved** 寄存器（如 `ra`, `sp`, `s0-s11`），而不需要保存所有的寄存器。
3. RISC-V 64位系统的寄存器大小为 **8 Byte**，在汇编代码读写内存的时候，偏移量是依次增加 8。

2. Pthread

实现思路

在 `put` 和 `get` 函数各自对于哈希表操作的 critical section 代码前后加锁，并且对于各个桶使用独立的锁。

修改的文件

- `notxv6/ph.c`

code

在 `ph.c` 中为每个桶初始化互斥锁：

```
pthread_mutex_t mlock[NBUCKET];
void init_lock() {
    for (int i=0; i<NBUCKET; i++) pthread_mutex_init(&mlock[i], NULL);
}
```

在 `put` 和 `get` 中，根据 `key` 定位到对应的桶，并在操作前后加锁与解锁：

```
static void put(int key, int value) {
    int i = key % NBUCKET;
    lock(i);
    // critical section
    unlock(i);
}

static struct entry* get(int key) {
    int i = key % NBUCKET;
    lock(i);
    // critical section
    unlock(i);
    return e;
}
```

调试过程中遇到的问题

1. 保证锁的粒度尽可能细：针对数组中的每个bucket分别使用一个独立的 `pthread_mutex_t`，能大幅减少多线程散列到不同桶时的锁竞争，提高并行效率。

导致并发出错的情况

```
static void
insert(int key, int value, struct entry **p, struct entry *n)
{
    struct entry *e = malloc(sizeof(struct entry));
    e->key = key;
    e->value = value;
    e->next = n;
    *p = e;
}
```

两个线程同时运行到 `insert`，并且插入的 `key` 均为 `x`，并且 `put` 调用 `insert` 时传入的 `p` 和 `n` 是相同的。于是先执行的线程1的 `e1->next` 指向原先的第一个元素，并且桶指向了 `e1`。后执行的线程2的 `e1->next` 也指向原先的第一个元素，并且桶指向了 `e2`。最终 `e1` 变成了野指针，线程1的数据没有被真正储存，并且产生内存泄露。

3. Barrier

实现思路

为了确保所有线程都能在同一点停下来同步，在 `barrier` 函数中利用条件变量及其配套的互斥锁来实现。需要管理当前到达屏障的线程数 `nthread` 和屏障轮次 `round`。对 `nthread` 的读写必须加锁。对于非本轮末位到达的线程，调用 `pthread_cond_wait` 等待广播通知；对于本轮末位到达的线程，负责重置当前轮次的到达线程数，增加轮次(`round`)，并发起广播，因此能够保证该轮最后一个线程执行后，前面所有挂起的线程才得以继续。

修改的文件

- `notxv6/barrier.c`

code

```
static void barrier()
{
    lock();
    bstate.nthread++;
    if (bstate.nthread == nthread) {
        // 最后一个到达的线程
        bstate.round++;
        bstate.nthread = 0;
        pthread_cond_broadcast(&bstate.barrier_cond);
    } else {
        // 等待其他线程到达
        pthread_cond_wait(&bstate.barrier_cond, &bstate.barrier_mutex);
    }
}
```

```
    unlock();  
}
```

调试过程中遇到的问题

1. 对共享变量 `bstate.nthread` 和 `bstate.round` 的读写要在 `lock()` 和 `unlock()` 包含的临界区内。
2. `pthread_cond_wait` 会在内部使当前线程睡眠的同时自动释放锁，被唤醒时会重新获取锁，在函数执行结束前，要调用 `unlock()` 释放锁，防止死锁。